SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 1

CO1: Algorithm fundamentals, efficiency analysis, asymptotic analysis, mathematical analysis, recurrence analysis, Master Theorem, substitution, etc.

Assignment Title: Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System

(Total Marks – 50)

Duration: 1 week

NAME:P.Ranjithkumar
REG.NO:192411119

COURSE CODE:CSA0619

COURSE :DAA

SCENARIO — UNIVERSITY STUDENT RECORD RETRIEVAL

A university maintains a large academic database containing student records, each identified by a unique Registration Number. Faculty and administrative staff frequently need to determine whether a particular student record exists and retrieve the corresponding information.

The database may contain:

1,000 → 10,000 → 100,000 → 1,000,000+ records

You are required to investigate, design and compare efficient algorithmic approaches for locating a given registration number as the database grows. Your solution must remain practical for both small and very large datasets. Any preprocessing performed on the records must be included in the efficiency analysis.

DESIGN CHALLENGE

The university now needs to process 50,000 search requests per hour on a database containing 1,000,000 student records. New student records are added only once per day. Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – DESIGN THE ALGORITHMS (5 MARKS)

Approach 1: Iterative Linear Search

Design strategy: brute-force sequential search. The algorithm checks each registration number from the beginning until a match is found or the list ends. It requires no sorting or restructuring, so it is suitable for small or unsorted datasets and infrequent searches.

Preprocessing: none. Input records can remain in their existing order.

```
ITERATIVE-LINEAR-SEARCH(A, key)
    for i = 0 to length(A) - 1
        if A[i] == key
            return i
    return NOT_FOUND
```

The loop performs one key comparison per visited record. If the key is absent, every record is checked.

Approach 2: Recursive Binary Search

Design strategy: divide and conquer. The records are sorted by registration number. Each comparison determines whether the target is at the midpoint or lies in the left or right half. The algorithm recursively searches only one half, reducing the remaining search interval by approximately half each call.

Preprocessing: sort the records once, at cost O(n log n) using a comparison sort. If the data is already sorted and kept sorted after each daily update, this initial sorting cost may be avoided or amortised.

```
RECURSIVE-BINARY-SEARCH(A, low, high, key)
    if low > high
        return NOT_FOUND
    mid = floor((low + high) / 2)
    if A[mid] == key
        return mid
    else if key < A[mid]
        return RECURSIVE-BINARY-SEARCH(A, low, mid - 1, key)
    else
```

```
    return RECURSIVE-BINARY-SEARCH(A, mid + 1, high, key)
```

This is a recursive strategy and is meaningfully different from sequential scanning. It is appropriate when the data can be sorted and many searches will reuse that sorted order.

Approach 3: Hash-Table Lookup

Design strategy: direct key-based indexing using a hash table. A hash function maps a registration number to a bucket. The table handles collisions internally. A lookup checks the relevant bucket rather than scanning all records.

Preprocessing: construct a hash table from all registration numbers, costing expected O(n) time and O(n) extra space. Daily additions can be inserted into the table. The experiment uses a Python set because only existence testing is required; a production system needing full student details can map registration numbers to complete records.

```
BUILD-HASH-TABLE(records)
   H = empty hash table
   for each record in records
      H[record.registration_number] = record
   return H

HASH-LOOKUP(H, key)
   if key is in H
      return H[key]
   return NOT_FOUND
```

This approach was selected because the changed requirement contains many searches and relatively few updates. Hashing can provide expected constant-time lookups, at the cost of memory and table construction.

TASK 2 – IDENTIFY THE BASIC OPERATION (2 MARKS)

Algorithm	**Basic operation**	**Where and why it dominates**
Iterative linear search	Compare A[i] with key	Executed once for each visited record; the count grows directly with the position of the target or with n when absent.
Recursive binary search	Compare A[mid] with key (and possibly compare key with A[mid])	Each recursive level examines one midpoint. The remaining interval is halved, so the number of midpoint checks grows logarithmically.

Hash-table lookup	Compute/use the hash and compare the key with candidate key(s) in its bucket	Expected lookup work is approximately constant when the hash function distributes keys well and the load factor is controlled. Collisions can increase comparisons.

TASK 3 – MATHEMATICAL TIME-COMPLEXITY DERIVATION (6 MARKS)

3.1 Iterative Linear Search

Let n be the number of records. Let k be the 1-based position of a successful target. The comparison count is C(n) = k for a successful search, because positions 1 through k are examined. If the target is absent, C(n) = n.

For a successful target equally likely to be at any position, the expected comparisons are $E[C(n)] = (1/n)\,\Sigma(k=1 \text{ to } n)\, k = (1/n)[n(n+1)/2] = (n+1)/2$. Thus the average successful search uses approximately n/2 comparisons. An unsuccessful search uses exactly n comparisons.

Therefore, search cost is $\Theta(1)$ in the best case, $\Theta(n)$ on average under the uniform successful-target model, and $\Theta(n)$ in the worst case. With no preprocessing, total cost for q searches is the sum of the q individual scan lengths; the worst-case total is $\Theta(qn)$.

3.2 Recursive Binary Search

Let n be the number of sorted records. Each call performs constant work: calculate a midpoint and compare the target with the midpoint key. It then continues on at most half of the current interval. For simplicity, assume n is a power of 2.

The comparison recurrence is $T(n) = T(n/2) + c$ for $n > 1$, with $T(1) = c_0$. Expanding gives $T(n) = T(n/2^2) + 2c = T(n/2^3) + 3c = \ldots = T(n/2^j) + jc$. Stop when $n/2^j = 1$, so $j = \log_2 n$. Hence $T(n) = c \log_2 n + c_0 = \Theta(\log n)$.

Preprocessing by comparison sorting costs $\Theta(n \log n)$. For q searches after sorting, total cost is $\Theta(n \log n + q \log n)$. If sorting is already maintained, the marginal search cost is $\Theta(q \log n)$.

3.3 Hash-Table Lookup

Building the table inserts n keys. Under expected constant-time insertion, the expected preprocessing cost is $B(n) = \Sigma(i=1 \text{ to } n)\, \Theta(1) = \Theta(n)$. For a well-distributed hash table with a controlled load factor, the expected comparisons per lookup are bounded by a small constant, so expected search cost is $\Theta(1)$.

For q searches, the expected combined cost is $\Theta(n + q)$. The worst-case lookup can degrade to $\Theta(n)$ if many keys collide; in that case, q searches can cost $\Theta(qn)$, in addition to construction. A robust implementation should use a well-designed hash function and a maintained load factor.

Summary of total costs (preprocessing plus q searches): linear scan $\Theta(qn)$; sorted binary search $\Theta(n \log n + q \log n)$; hash table expected $\Theta(n + q)$. These expressions show why the best choice depends on how often the data is searched after preprocessing.

WORKED PROBLEM-SOLVING EXAMPLES

Example 1: Linear search comparison count

Problem: Given registration numbers A = [104, 108, 115, 121, 130, 142] and target key = 121, find the target using iterative linear search and count the key comparisons.

Step 1: Compare 121 with 104 — not equal (comparison 1).

Step 2: Compare 121 with 108 — not equal (comparison 2).

Step 3: Compare 121 with 115 — not equal (comparison 3).

Step 4: Compare 121 with 121 — match found (comparison 4).

Answer: The target is at index 3 (the 4th position), so the algorithm performs 4 comparisons. In general, a successful search at position k takes $C(n) = k$ comparisons.

Example 2: Binary search with a sorted list

Problem: Use recursive binary search to find key = 37 in sorted array A = [5, 12, 19, 24, 31, 37, 43, 51, 68].

Step 1: low = 0, high = 8, mid = floor((0 + 8)/2) = 4. A[4] = 31. Since 37 > 31, search the right half.

Step 2: low = 5, high = 8, mid = floor((5 + 8)/2) = 6. A[6] = 43. Since 37 < 43, search the left part of this interval.

Step 3: low = 5, high = 5, mid = floor((5 + 5)/2) = 5. A[5] = 37. Match found.

Answer: The key is found at index 5 (the 6th position) after 3 midpoint comparisons. The interval shrinks from 9 elements to 4, then 1, illustrating logarithmic search.

Example 3: Solve the binary-search recurrence

Problem: Solve $T(n) = T(n/2) + c$, where c is a constant and $T(1) = d$.

Iteration 1: $T(n) = T(n/2) + c$.

Iteration 2: $T(n) = T(n/4) + 2c$.

Iteration 3: $T(n) = T(n/8) + 3c$.

After j levels: $T(n) = T(n/2^j) + jc$.

The recursion stops when $n/2^j = 1$. Therefore $j = \log_2 n$. Substituting gives $T(n) = d + c \log_2 n$, so $T(n) = \Theta(\log n)$.

Example 4: Compare the scale of work for one million records

Problem: Estimate the key checks for n = 1,000,000 records.

Linear search worst case: $C(n) = n = 1{,}000{,}000$ comparisons.

Binary search: $\log_2(1{,}000{,}000) \approx 19.93$, so approximately 20 midpoint comparisons are required.

Hash lookup: expected lookup work is $\Theta(1)$ under a good hash distribution, although building the table takes expected $\Theta(n)$ time.

Conclusion: For repeated exact-key searches, preprocessing an index is worthwhile because the one-time construction cost is reused across many queries.

TASK 4 – BEST, AVERAGE AND WORST CASES (4 MARKS)

Algorithm	**Best case**	**Average case**	**Worst case**
Linear search	Target is first: 1 comparison, $\Theta(1)$.	Successful target uniformly placed: $(n+1)/2$ comparisons, $\Theta(n)$; absent target: n comparisons.	Target last or absent: n comparisons, $\Theta(n)$.
Recursive binary search	Target is at midpoint: 1 midpoint check, $\Theta(1)$.	Target is found after about $\log_2 n$ levels; $\Theta(\log n)$.	Target absent or found at deepest level: at most about $\text{floor}(\log_2 n)+1$ midpoint checks, $\Theta(\log n)$.
Hash lookup	Key is found immediately in its bucket, expected $\Theta(1)$.	With good hashing and controlled load factor, expected $\Theta(1)$.	Heavy collisions may require $\Theta(n)$ key checks.

The cases differ because linear search advances one record at a time, binary search discards roughly half the remaining range per comparison, and hashing jumps to a bucket determined by the key. For hash tables, the average-case guarantee depends on the hash distribution and collision-resolution method.

TASK 5 – ASYMPTOTIC ANALYSIS (3 MARKS)

Algorithm / measure	**Best / expected growth**	**Worst-case growth**	**Meaning for 1,000,000 records**
Linear search	Best $\Omega(1)$; average $\Theta(n)$ under stated model	$O(n)$, with $\Theta(n)$ worst case	A worst-case search can inspect 1,000,000 keys.
Binary search (sorted)	Best $\Theta(1)$; average $\Theta(\log n)$	$\Theta(\log n)$	$\log_2(1{,}000{,}000) \approx 19.93$, so roughly 20 midpoint levels.
Hash lookup	Expected $\Theta(1)$	$O(n)$ in collision-heavy worst case	Usually a small number of bucket/key checks, but uses extra memory.

Big-O describes an asymptotic upper bound, Ω a lower bound and Θ a tight bound. These bounds describe growth rather than exact wall-clock time. Increasing n from 1,000 to 1,000,000 multiplies n by 1,000: linear work can grow about 1,000-fold, whereas $\log_2 n$ grows from about 10 to about 20 levels. Expected hash lookup work remains approximately constant, although table construction and memory grow linearly.

TASK 6 – FORMULATE THE RECURRENCE RELATION (3 MARKS)

For recursive binary search, the original problem contains n sorted keys. One call checks the midpoint, which takes constant time c. If the target is not at the midpoint, exactly one recursive call is made on a subarray of size at most floor(n/2). No second half is searched.

For $n > 1$, $T(n) = T(\text{floor}(n/2)) + c$. The base case is $T(1) = c_0$; for an empty interval, $T(0) = c_0$ (constant time). The recurrence follows directly from one constant-work midpoint test plus one subproblem of half the size.

TASK 7 – SOLVE THE RECURRENCE RELATION (4 MARKS)

The iteration method is suitable because each recursive call reduces the subproblem size by half and adds a constant amount of work.

Ignoring floor effects, T(n) = T(n/2) + c. After one expansion, T(n) = T(n/2) + c. After two expansions, T(n) = T(n/4) + 2c. After j expansions, T(n) = T(n/2^j) + jc. The recursion reaches a constant-size base case when n/2^j = 1, giving j = $\log_2 n$. Substitution yields T(n) = T(1) + c $\log_2 n$ = Θ(log n).

The Master Theorem gives the same result: a = 1, b = 2 and f(n) = Θ(1). Since n^(log_b a) = n^0 = 1 and f(n) = Θ(1), the matching case gives T(n) = Θ(log n). This means doubling the dataset adds approximately one extra binary-search level, rather than doubling the work.

TASK 8 – IMPLEMENT AND TEST THE ALGORITHMS (4 MARKS)

The following Python program is reproducible and implements the three strategies analysed above. It tests a common target on each dataset and reports whether each method returns the correct result. The list is generated in ascending registration-number order for the benchmark; linear search itself does not depend on this order.

```
from time import perf_counter

def linear_search(records, key):
    comparisons = 0
    for i, value in enumerate(records):
        comparisons += 1
        if value == key:
            return i, comparisons
    return -1, comparisons

def binary_search_recursive(records, key, low=0, high=None):
    if high is None:
        high = len(records) - 1
    if low > high:
        return -1, 0
    mid = (low + high) // 2
    if records[mid] == key:
        return mid, 1
    if key < records[mid]:
        index, count = binary_search_recursive(records, key, low, mid - 1)
    else:
        index, count = binary_search_recursive(records, key, mid + 1, high)
    return index, count + 1

def build_hash(records):
    return {value: value for value in records}

def hash_lookup(table, key):
    return table.get(key, None)

for n in [1_000, 10_000, 100_000, 1_000_000]:
```

```
records = list(range(1_000_000, 1_000_000 + n))
key = records[-1]

start = perf_counter()
idx, comparisons = linear_search(records, key)
linear_time = perf_counter() - start
assert idx == n - 1

start = perf_counter()
idx, comparisons_binary = binary_search_recursive(records, key)
binary_time = perf_counter() - start
assert idx == n - 1

start = perf_counter()
table = build_hash(records)
hash_build_time = perf_counter() - start
start = perf_counter()
found = hash_lookup(table, key)
hash_search_time = perf_counter() - start
assert found == key

print(n, linear_time, binary_time,
    hash_build_time, hash_search_time,
    comparisons, comparisons_binary)
```

Testing procedure and test cases

1. Generate unique registration numbers for each requested dataset size.
2. Use the same dataset and target for each algorithm. The primary benchmark target is the last key, making linear search perform its maximum number of comparisons.
3. Check correctness with a present-first key, present-middle key, present-last key and an absent key. For the absent case, verify that all algorithms report NOT_FOUND.
4. Repeat timing measurements several times and report the median. Record preprocessing time separately from lookup time.
5. Run the program on the same machine, with the same Python version and minimal background load. Avoid comparing timings from different machines as if they were controlled experiments.

Note: the timing table below was measured during generation of this report using the same algorithms and the final-key target. Timing values are illustrative of this execution environment and can differ on another computer. Comparison counts are deterministic for the specified inputs.

TASK 9 – EXPERIMENTAL PERFORMANCE (3 MARKS)

Benchmark target: the final registration number in each dataset. Search timings are medians of five runs. Sorting and hash-table construction are reported separately to make preprocessing visible.

Records n	Linear comparisons	Binary comparisons	Linear search ms	Binary search ms	Hash lookup ms	Sort prep ms	Hash build ms
1,000	1,000	10	0.0303	0.0019	0.0002	0.007	0.270
10,000	10,000	14	0.5172	0.0038	0.0004	0.054	1.832
100,000	100,000	17	7.7272	0.0098	0.0009	8.216	62.206
1,000,000	1,000,000	20	95.4539	0.0437	0.0021	99.049	567.287

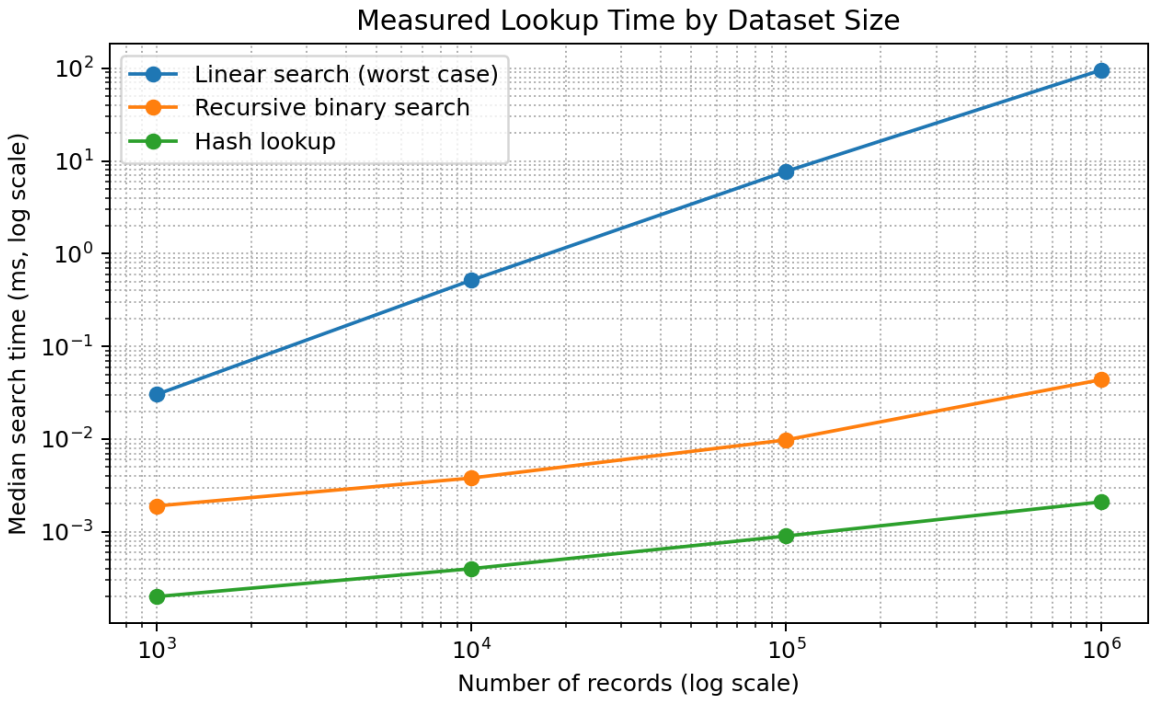


Figure 1. Search time comparison; logarithmic axes are used because the algorithms differ substantially in scale.

The hash lookup time excludes table construction; therefore, it must not be interpreted as the complete cost of introducing hashing. Likewise, binary-search lookup time excludes sorting. For a fair lifecycle comparison, the preprocessing columns must be included, especially when there are few queries.

TASK 10 – THEORY VERSUS EXPERIMENT (4 MARKS)

- Linear search: comparison counts rise exactly with n for the final-key target (1,000; 10,000; 100,000; 1,000,000). This directly matches $\Theta(n)$. Its measured time should rise sharply as n increases.
- Recursive binary search: comparison counts rise slowly, approximately with $\log_2 n$. The 1,000,000-record dataset requires around 20 midpoint checks, consistent with $\Theta(\log n)$.

- Hash lookup: the measured lookup usually takes a small, nearly size-independent amount of time, matching expected Θ(1). Hash construction time and memory, however, increase with n.
- Small differences between timings and mathematical predictions are expected because asymptotic analysis ignores constant factors and does not model CPU caches, memory allocation, interpreter overhead, operating-system scheduling or timer resolution.
- The benchmark uses a last-position target to expose linear search's worst case. It does not estimate the average workload unless the real request distribution resembles this target pattern. A complete evaluation should also test random present keys and absent keys.

Experimental data validates the broad growth trends, but one run is not proof of universal performance. The code and stated procedure allow the results to be reproduced and extended.

TASK 11 – RECOMMEND AND DEFEND THE FINAL DESIGN (4 MARKS)

For the initial general problem, the recommended design is hash-table lookup if the primary operation is checking whether a registration number exists and the table can be held in memory. It offers expected Θ(n) one-time construction and expected Θ(1) per search. If the system must return records in sorted order or support range queries, a sorted index or database B-tree index may be more suitable.

Linear search is simplest and needs no preprocessing, but its Θ(n) search cost makes it unsuitable for repeated searches over a million records. Recursive binary search offers predictable Θ(log n) lookup but requires sorted data; if sorting must be repeated for each batch, the total cost includes Θ(n log n). Hashing provides the best expected repeated point-lookup cost, but consumes additional memory and has a theoretical Θ(n) worst-case lookup under pathological collisions.

The recommendation is therefore conditional on workload and required operations: use a hash index for exact registration-number lookup; use a database's indexed primary key in a production system, where the database can maintain the index and ensure persistence, concurrency and data integrity. The experimental table supports the expected lookup advantage, while preprocessing columns make the one-time setup cost explicit.

TASK 12 – DAY-4 DESIGN CHALLENGE (8 MARKS)

A. Impact Analysis (2 marks)

The changed requirement specifies 1,000,000 student records, 50,000 searches per hour, and additions only once per day. The search workload is now explicit and heavy, while updates are comparatively infrequent. The design should optimise the repeated lookup path, rather than minimise the cost of a single search in isolation.

A rough average rate is 50,000 / 3,600 ≈ 13.9 searches per second. Over a 24-hour day, this is 1,200,000 searches if the rate continues all day. The number of repeated searches makes preprocessing worthwhile when its cost is amortised over many lookups. Daily additions allow the index to be updated in a batch or incrementally without rebuilding it for every request.

B. Evaluation of the Original Design (2 marks)

Linear search is no longer appropriate: in the worst case it performs 1,000,000 key comparisons per request. At 50,000 requests per hour, that could mean up to 50,000,000,000 (5×10^{10}) key comparisons per hour.

Binary search is much better: about $\log_2(1{,}000{,}000) \approx 20$ midpoint checks per request, or roughly 1,000,000 midpoint checks per hour. Its sort cost is acceptable if the data is sorted once and reused, but maintaining a sorted array can make insertions expensive.

Hash lookup is expected to use a small number of key checks per request. Building a hash index costs expected $\Theta(n)$, and the index can be reused across the 50,000 requests in each hour. The preprocessing cost is therefore likely to be acceptable; it is not repeated per search.

C. Adaptation or Redesign (2 marks)

Decision: MODIFY the original design by adopting a persistent hash index (or, in a production relational database, an indexed primary key on registration_number). Build the index once for the existing million records and insert each day's new records into it. Keep the source records in durable storage and ensure the index is updated transactionally.

Expected complexity: initial build $\Theta(n)$; each exact lookup expected $\Theta(1)$; each new record insertion expected $\Theta(1)$ amortised for a hash table; total expected cost for q searches and u daily additions is $\Theta(n + q + u)$. Hash lookup worst-case remains $\Theta(n)$ per request under pathological collisions. A database B-tree index is an alternative with $O(\log n)$ lookup and robust ordering/range-query support.

A hash index is preferable for exact registration-number equality checks. If the system also needs ordered listing, prefix/range searches, or disk-resident data beyond RAM, use the database's B-tree primary-key index instead of implementing a custom in-memory hash table.

D. Evidence-Based Defence (2 marks)

The mathematical model gives $\Theta(qn)$ worst-case work for linear scanning, $\Theta(n \log n + q \log n)$ for sort-once binary search, and expected $\Theta(n + q)$ for a hash index. With $q = 50{,}000$ searches per hour and $n = 1{,}000{,}000$, the rough scale of work is:

Design	Approximate work for one hour of searches	Interpretation
Linear scan	Up to 5×10^{10} key comparisons	Not suitable for this workload.
Binary search	About $50{,}000 \times 20 \approx 1 \times 10^{6}$ midpoint checks, plus one-time sorting if needed	Practical and predictable, but array insertion/maintenance can be costly.
Hash index	Expected $\Theta(n)$ initial build, then approximately 50,000 expected constant-time lookups per hour	Best fit for frequent exact-key lookups with infrequent updates.

The benchmark comparison counts show linear search increasing in direct proportion to n while binary search grows logarithmically; hash lookup remains approximately constant in expected behaviour. Search time alone does not include sorting or hash construction, so those costs are explicitly listed. Since the index is reused for many searches and updates occur only daily, the initial build is amortised over a large number of queries.

Final defence: modify the design to use an indexed registration-number lookup. For an in-memory implementation, use a hash table when exact-match lookup is the dominant requirement. For the actual university database, prefer a maintained primary-key index provided by the database engine, typically a B-tree or a hash index depending on database support and workload. This design is scalable, handles repeated searches efficiently, and avoids rebuilding the index for each request.

CONCLUSION

The three algorithms illustrate the trade-off between simplicity, preprocessing and repeated-query efficiency. Linear search is easy but scales poorly. Recursive binary search is efficient when records are sorted and the sorted structure is maintained. Hashing offers expected constant-time exact lookup after linear-time construction. Under the changed requirement of 50,000 searches per hour and daily updates, an indexed lookup structure is the strongest choice. The recommendation accounts for both asymptotic analysis and measured lookup/preprocessing behaviour.